

COLLAGE OF COMPUTING AND INFORMATICS SCIENCE

COURSE TITLE: REQUIREMENT ENGINEERING

COURSE CODE: SEng5131

PROJECT TITLE : PERSONAL BUDGET TRACKER-INCOME/EXPENCE

Submitted by: **GROUP 1**

No	*GROUP NAME*	*ID*
1	BEWKET AFEWORK	MAU1600290
2	SENAIT KEDRU	MAU1601084
3	HAILEMARYAM EYASU	MAU1600639
4	KIDANU AMARE	MAU1600740
5	TADESSE TILAHUN	MAU1601128
6	YARED GIDEY	MAU1601295
7	MULUGETA LIUL	MAU1600957

Submitted to: **Mr. Banteamlak.B**

Department : SOFTWARE ENGINNERING

MAU

YEAR : 3rd

Program: Bachelor's Degree

15/04/2018 E.C

TULU AWULIYA

ETHIOPIA

Approval Sheet

This document is submitted in partial fulfillment of the requirements for the course **Requirement Engineering SEng5131**.

Approved by:

Name	***Role***	***Signature***	***Date***
Adisu A.	Project Advisor	_____	//____
Mr. Banteamlak.A	Course Instructor	_____	//____

Acknowledgements

We would like to express our sincere gratitude to our advisor **Mr. Adisu A.** for his continuous guidance, support, and valuable feedback throughout the development of this project. His expertise and encouragement were instrumental in shaping our work.

We also extend our thanks to our course instructor **Mr. Banteamlak B.** for providing us with the foundational knowledge in Requirement Engineering and for his insightful reviews of our documentation.

Our appreciation goes to the **College of Computing and Informatics Science** at MAU for providing the resources and environment necessary for this academic endeavor.

Finally, we thank our families and friends for their patience and support during this project.

Table of Contents

Titles **Pages**

Chapter 1: Introduction.....1

1.1 Purpose.....1
1.2 Document Conventions.....1
1.3 Intended Audience and Reading Suggestions.....1
1.4 Background and Context.....3
1.5 Statement of the Problem.....3
1.6 System Objectives.....3
1.6.1 General Objective.....3
1.6.2 Specific Objectives.....3
1.7 Project Scope.....4
1.8 Significance of The Project.....4
1.9 Methodology.....4
1.10 Feasibility Study.....4
1.10.1 Technical Feasibility.....4
1.10.2 Economic Feasibility.....5
1.10.3 Schedule Feasibility.....5

Chapter 2: Overall Description.....6
2.1 Product Perspective.....6
2.2 Product Features.....6
2.3 User Classes and Characteristics.....7
2.4 Operating Environment.....7
2.5 Design and Implementation Constraints.....7
2.6 User Documentation.....7
2.7 Assumptions and Dependencies.....7

Chapter 3: System Requirements.....8

3.1 Functional Requirements.....8
3.2 Non-Functional Requirements.....10
3.2.1 Usability Requirements.....10
3.2.2 Security Requirements.....10
3.2.3 Performance Requirements.....10
3.2.4 Reliability Requirements.....11
3.2.5 Maintainability Requirements.....11
3.3 Other Requirements.....11
3.3.1 Business Requirements.....11
3.3.2 User Requirements.....11
3.3.3 Legal and Regulatory Requirements.....12
3.3.4 Documentation Requirements.....12
3.3.5 Training Requirements.....12
3.3.6 Quality Requirements.....12

Chapter 4: Design Analysis.....14

4.1 Use Case Modeling……14
4.1.1 Use Case Descriptions……14
4.2 Structural Design……21
4.3 Behavioral Design……22
4.3.1 Sequence Diagrams……23
4.3.2 Activity Diagrams……24
4.4 Traceability Matrix……27

Chapter 5: External Interface Requirements……29

5.1 User Interfaces……29
5.2 Hardware Interfaces……30
5.3 Software Interfaces……33
5.4 Communications Interfaces……33

Appendix A: Glossary……35
Appendix B: Issues List……36

References……37

List of Figures……III
List of Tables……IV
Abstract……V
Abbreviations……VI

List of Figures

Figure Pages

Figure 4.1: Use Case Diagram.....14
Figure 4.2: Class Diagram.....21
Figure 4.3: Login Sequence Diagram.....22
Figure 4.4: Add Income Sequence Diagram.....23
Figure 4.5: Add Expense Sequence Diagram.....23
Figure 4.6: View Budget vs Actual Sequence Diagram.....24
Figure 4.7: View Reports Sequence Diagram.....24
Figure 4.8: User Registration & Login Activity Diagram.....25
Figure 4.9: Add Income Activity Diagram.....25
Figure 4.10: Manage Categories Activity Diagram.....26
Figure 4.11: Set Budget & View Progress Activity Diagram.....26
Figure 4.12: Generate & Export Report Activity Diagram.....27
Figure 4.13: Logout Activity Diagram.....27
Figure 5.1: Register Interface.....28
Figure 5.2: Login Interface.....29
Figure 5.3: Dashboard Interface.....29
Figure 5.4: Add Income Interface.....30
Figure 5.5: Add Expense Interface.....31
Figure 5.6: Category Interface.....31
Figure 5.7: Reports Interface.....32
Figure 5.8: Budget Interface.....32

List of Tables

Table Pages

Table 1.1: Technical Feasibility Analysis.....5
Table 1.2: Economic Feasibility Breakdown.....5
Table 1.3: Schedule Feasibility Timeline.....5
Table 2.1: Feature Overview.....6
Table 3.1: Functional Requirements Specification.....8
Table 4.1: Use Case UC-01: Register User.....14
Table 4.2: Use Case UC-02: Login User.....15
Table 4.3: Use Case UC-03: Add Income.....15
Table 4.4: Use Case UC-04: Add Expense.....16
Table 4.5: Use Case UC-05: Add Category.....16
Table 4.6: Use Case UC-06: Edit/Delete Category.....17
Table 4.7: Use Case UC-07: Set Budget.....18
Table 4.8: Use Case UC-08: View Budget Progress.....18
Table 4.9: Use Case UC-09: View Reports.....19
Table 4.10: Use Case UC-10: Export PDF Report.....19
Table 4.11: Use Case UC-11: Logout.....20
Table 4.12: Use Case UC-12: View Dashboard.....20
Table 4.13: Requirements Traceability Matrix.....28
Table A.1: Terminology Definitions.....34

Abstract

The **Personal Budget Tracker System** is a web-based system designed to help individuals manage their personal finances effectively. It provides a secure, intuitive, and offline-capable platform for tracking income and expenses, setting budgets, generating visual reports, and exporting financial summaries. Developed using open-source technologies (PHP, MySQL, HTML5, CSS3, JavaScript), the system emphasizes data privacy, cost-free access, and ease of use. This project follows IEEE 830-1993 standards for requirement specification and employs a hybrid development methodology combining Waterfall and iterative prototyping. The system aims to bridge the gap in the current market by offering a privacy-first, subscription-free alternative to existing financial tools, thereby promoting financial literacy and responsible budgeting.

Abbreviations

Abbreviations	Expanstions
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CSS	Cascading Style Sheets
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IEEE	Institute of Electrical and Electronics Engineers
JS	JavaScript
MVC	Model-View-Controller
NFR	Non-Functional Requirement
OWASP	Open Web Application Security Project
PDO	PHP Data Objects
PDF	Portable Document Format
SRS	Software Requirements Specification
SQL	Structured Query Language
SUS	System Usability Scale
TCP/IP	Transmission Control Protocol/Internet Protocol
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
XSS	Cross-Site Scripting

Chapter 1: Introduction

1.1 Purpose

This Software Requirements Specification document serves as the authoritative foundation for the Personal Budget Tracker System development project. Following **IEEE 830-1993** standards, it establishes comprehensive guidelines that bridge stakeholder expectations with technical implementation. The document facilitates clear communication among project participants, provides measurable criteria for system validation, and ensures alignment between user needs and delivered functionality throughout the software development lifecycle.

1.2 Document Conventions

This document adheres to specific formatting conventions to ensure clarity and consistency:

1. **Requirement Identification**: Functional requirements are labeled as **FR-XX** (e.g., FR-01, FR-02), while non-functional requirements use **NFR-XX** (e.g., NFR-01, NFR-02). Each requirement includes priority designation (High, Medium, Low).
2. **Cross-Referencing**: Internal references to sections and figures follow the format "Section X.Y" or "Figure X.Y." All references are hyperlinked in the digital version for easy navigation.
3. **Diagrams and Figures**: UML diagrams and interface mock-ups are sequentially numbered (Figure 2.1, Figure 3.1, etc.) with descriptive captions. All diagrams follow Object Management Group UML 2.5 standards.

Typography Conventions:

I. **Bold** text indicates key terms or interface components.
II. *Italic* text emphasizes important concepts or alternative terms
III. **Quoted** text indicates user interface labels or system messages
IV. **Table Conventions**: All tables include sequential numbering (Table 1.1, Table 2.1) with clear column headers and row definitions. Tables use alternating row shading for improved readability.
V. **Priority Indicators**: Requirements include priority levels:
    1. High: Critical functionality without which the system cannot operate.
    2. Medium: Important functionality that enhances system value.
    3. Low: Desirable functionality that provides additional user benefits.
VI. **Version Control**: Document versions follow semantic versioning (Major.Minor.Patch) with change logs maintained in a separate appendix.
VII. **Measurement Units**: Time measurements use 24-hour format (HH:MM) and dates follow ISO 8601 standard (YYYY-MM-DD).

1.3 Intended Audience and Reading Suggestions

This document serves multiple stakeholders with varying technical expertise and project involvement levels:

Primary Audiences:

- **Project Sponsors & Clients** are concerned with business value, ROI, compliance, and scope. They should focus on Chapters 1, 2, and 6, with an executive summary reading approach.
- **Project Managers** focus on timelines, resources, risks, and milestones. They should comprehensively read Chapters 1, 1.8, and 6, along with Appendix C.
- **Software Developers** require technical specifications, architecture, and interface details. They should review Chapters 3, 4, 5, and Appendix B in detail.
- **Quality Assurance Teams** focus on testability, validation criteria, and edge cases. They should concentrate on Chapters 3 and 3.2, and Chapter 6 for test case development.
- **System Architects** are concerned with system structure, integration, and scalability. They should review Chapter 4 and Appendix B with emphasis on diagrams and models.
- **UI/UX Designers** need interface requirements and user workflows. They should focus on Chapter 5 and the activity diagrams.
- **Academic Evaluators** assess methodology compliance and documentation standards. They should read all chapters and appendices comprehensively.
- **End Users** (for reference) focus on system capabilities and features. They may review Chapters 2 and 5 to understand functionality.

Reading Path Recommendations:

1. **For Complete Understanding**: Read sequentially from Chapter 1 through Chapter 6, reviewing appendices for supplementary information.
2. **For Technical Implementation Focus**: Begin with Chapter 3 (Requirements), proceed to Chapter 4 (Design), then Chapter 5 (Interfaces), and reference Appendix B (Models) as needed.
3. **For Project Management Focus**: Start with Chapter 1 (Introduction), review Chapter 1.8 (Feasibility), examine Chapter 6 (Other Requirements), and monitor Appendix C (Issues).
4. **For Quality Assurance Focus**: Concentrate on Chapter 3 (Requirements), particularly Section 3.2 (Non-Functional Requirements), with reference to test case development guidelines.
5. **For Academic Review Focus**: Evaluate complete document structure against IEEE 830-1993 standards, assessing requirement traceability and documentation completeness.

Document Navigation Aids:

- **Table of Contents**: Provides hierarchical document structure with page references.
- **Index of Figures/Tables**: Lists all visual elements with location information.
- **Glossary (Appendix A)**: Defines technical terms and acronyms.
- **Cross-Reference Matrix**: Shows relationships between requirements, designs, and tests.
- **Hyperlinks**: Digital version includes navigation links between related sections.

Prerequisite Knowledge:

1. Basic understanding of personal finance concepts (income, expenses, budgeting)
2. Familiarity with web application terminology.
3. Awareness of basic security principles for online systems.

1.4 Background and Context

Developed as the capstone project for the Requirement Engineering course, this system addresses the critical need for accessible personal finance management tools. The project emerged from extensive analysis revealing significant limitations in existing solutions, particularly concerning data privacy, economic accessibility, and user experience. Current market offerings either compromise user privacy through cloud data storage or impose recurring financial burdens through subscription models, creating a distinct gap for a privacy-first, cost-free alternative that maintains professional functionality.

1.5 Statement of the Problem

The current landscape for personal finance management tools is inadequate for a significant portion of users, characterized by:

1. **Inefficiency of Manual Systems:** Paper-based ledgers and static spreadsheets lack automation, making data aggregation and analysis difficult and unreliable. They provide no proactive alerts and are vulnerable to loss.
2. **Economic and Privacy Barriers:** Many commercial applications operate on a subscription model, creating an ongoing cost. More critically, they often require cloud synchronization, posing a significant privacy risk by storing sensitive financial data on third-party servers.
3. **Usability Issues:** Free or open-source alternatives frequently suffer from complex interfaces and steep learning curves, discouraging consistent use.
4. **Platform Limitations:** Mobile-centric applications can be inefficient for detailed data entry and comprehensive reporting due to screen size constraints.

Our analysis indicates a clear need for a system that is **powerful, private, simple to use, and free of charge**—a gap the Personal Budget Tracker System aims to fill.

1.6 System Objectives

1.6.1 General Objective

To design and develop a secure, intuitive, and fully functional web-based personal budget tracking system that enables effective financial monitoring, analysis, and planning while ensuring complete data privacy, offline functionality, and zero financial cost to end-users.

1.6.2 Specific Objectives:

- Implement robust user authentication using bcrypt hashing algorithm.
- Develop comprehensive interfaces for income and expense transaction management.
- Create dynamic category management with user-defined Income/Expense types.
- Provide budget planning with real-time comparisons and overspending warnings.
- Integrate graphical reporting capabilities across multiple timescales.
- Implement PDF export functionality for financial summaries.
- Ensure consistent UI/UX across all application modules.
- Embed security best practices throughout the application stack.
- Conduct thorough usability testing with System Usability Scale targets.

1.7 Project Scope

In-Scope Features:

- **Primary user interfaces** (Register, Login, Dashboard, Income, Expense, Category, Reports, Budget, Logout).
- **Core financial operations** including transaction recording and categorization.
- **Security measures** including password hashing and SQL injection prevention.
- **Data portability** through PDF export functionality.

Out-of-Scope Features:

- Multi-user or collaborative budgeting capabilities.
- External financial integrations with banks or payment systems.
- Native mobile application development.
- Advanced financial advisory functions.
- Cloud infrastructure or scalability services.

1.8 Significance of The Project

This project demonstrates academic mastery of requirement engineering principles while delivering practical value through a privacy-centric financial management tool. It promotes financial literacy, addresses underserved user needs for offline functionality, and provides a cost-free alternative to commercial solutions without compromising data sovereignty.

1.9 Methodology

A hybrid methodology combining structured Waterfall phases with iterative prototyping was employed. The development process encompassed requirements gathering through user interviews and surveys, system modeling with UML diagrams, architectural design following MVC patterns, iterative implementation with module-by-module development, and parallel documentation following IEEE standards.

Development Tools and Technologies:

1. **Frontend**: **HTML5**, **CSS3**, **JavaScript**, **Chart.js**
2. **Backend**: **PHP** 8.2 with **PDO** database access
3. **Database**: **MySQL** 8.0 with **InnoDB** storage engine
4. **Security**: bcrypt hashing, prepared statements, session management
5. **Modeling**: draw.io for UML diagrams and interface wire frames
6. **Environment**: **XAMPP** stack for local development

1.10 Feasibility Study

Feasibility: The assessment of whether a project can be successfully completed given technical, economic, and schedule constraints, evaluating practicality and viability before committing resources.

1.10.1 Technical Feasibility

Technical Feasibility: Evaluation of whether the proposed system can be developed with available technology, expertise, and resources, considering hardware, software, and technical capabilities.

Table 1.1: Technical Feasibility Analysis

Component	Technology	Capability Assessment	Risk Level
Authentication	PHP bcrypt	Proven password security	Low
Data Processing	PHP/MySQL	Efficient transaction handling	Low
Chart Generation	Chart.js	Responsive visualizations	Low
PDF Export	TCPDF	Reliable document generation	Medium
Security	OWASP standards	Comprehensive protection	Low

1.10.2 Economic Feasibility

Economic Feasibility: Analysis of whether the project makes economic sense, considering costs, benefits, market needs, and potential return on investment from a commercial perspective.

Table 1.2: Economic Feasibility Breakdown

Cost Category	Estimated Cost	Justification
Development Labor	0	Academic contribution
Software Tools	0	Open-source solutions
Testing Resources	0	University facilities
Optional Hosting	600/year	Basic shared hosting
Total	600 (optional)	Minimal investment

1.10.3 Schedule Feasibility

Schedule Feasibility: Assessment of whether the project can be completed within the available time frame, considering task dependencies, resource availability, and project complexity.

Table 1.3: Schedule Feasibility Timeline

Phase	Duration	Key Deliverables	Success Criteria
Requirements	1 week	Complete SRS document	Stakeholder approval
System Design	1 week	UML diagrams, UI mockups	Design review passed
Implementation	2 weeks	Functional codebase	Module integration
Testing	1 weeks	Comprehensive test reports	Bug resolution
Documentation	1 week	Final SRS, user manual	Complete documentation
Total	6 weeks	Complete system	Project delivery

Chapter 2: Overall Description

2.1 Product Perspective

The Personal Budget Tracker System operates as a self-contained web application employing client-server architecture with clear separation between presentation, business logic, and data layers. The system maintains complete independence from external financial services, ensuring user privacy and operational reliability without dependencies on third-party APIs or cloud synchronization services.

The architecture supports local deployment on standard web hosting environments or personal computing setups using integrated solutions like XAMPP, providing flexibility for diverse user environments while maintaining consistent functionality across deployment scenarios.

2.2 Product Features

Table 2.1: Feature Overview

Feature Category	Specific Feature Implementation	User Benefit Realization
User Account Management	Secure registration with email validation, password-based authentication with bcrypt hashing, individual account isolation	Protected financial data access with personalized experience
Transaction Recording	Income and expense entry with date selection, amount specification, category assignment, optional descriptive notes	Complete financial activity tracking with contextual information
Categorization System	Custom category creation with Income/Expense type designation, category editing and deletion with transaction reassignment	Personalized financial organization adapting to individual needs
Budget Planning	Monthly budget allocation for specific expense categories, budget amount specification with validation	Proactive financial control with spending limitation establishment
Progress Monitoring	Real-time comparison between budgeted amounts and actual expenditures, visual progress indicators with warning triggers	Immediate feedback on financial discipline with overspending alerts
Reporting Capabilities	Graphical visualization of financial data through charts and graphs, time-based filtering with period selection	Insight generation through data transformation into understandable formats
Data Export Functionality	PDF document generation with financial summaries, chart inclusion with professional formatting	Data portability for external analysis and documentation

2.3 User Classes and Characteristics

Primary User: Individual adults managing personal finances with basic computer literacy. The system assumes no financial management expertise but requires willingness to regularly record transactions. Users typically seek improved financial control without complex tools or ongoing costs.

2.4 Operating Environment

Server Requirements:

- ✓ **PHP** 8.2 or higher with **PDO** extension
- ✓ **MySQL** 8.0 or compatible database
- ✓ Apache web server or equivalent

Client Requirements:

- ✓ Modern web browser (Chrome 80+, Firefox 75+, Edge 80+)
- ✓ JavaScript enabled
- ✓ Minimum screen resolution: 1024×768 pixels
- ✓ Recommended internet connection for initial access

2.5 Design and Implementation Constraints

- ❖ **Privacy Constraint**: All user data must remain locally stored without cloud synchronization
- ❖ **Cost Constraint**: Zero monetary cost for end-users throughout system lifecycle
- ❖ **Accessibility Constraint**: Must function without persistent internet connectivity
- ❖ **Security Constraint**: Must implement industry-standard security practices
- ❖ **Technology Constraint**: Must utilize open-source technologies exclusively

2.6 User Documentation

The system includes integrated help documentation with tooltips, form validation messages, and error guidance. A comprehensive user manual provides step-by-step instructions for all features, while context-sensitive help appears during critical operations like budget setting or report generation.

2.7 Assumptions and Dependencies

Key Assumptions:

1. Users maintain regular access to a computer or mobile device with web browsing capability
2. Users possess basic understanding of personal finance concepts
3. Users will consistently record transactions for accurate tracking
4. Deployment environment provides PHP and MySQL support

Critical Dependencies:

1. PHP environment with bcrypt support
2. MySQL database with InnoDB engine
3. Web server supporting HTTPS for secure transmission
4. Client browsers supporting modern JavaScript features

Chapter 3: System Requirements

3.1 Functional Requirements

Functional Requirements: Specifications describing what the system must do, including behaviors, functions, and features that users can directly observe and interact with during system operation.

Table 3.1: Functional Requirements Specification

ID	Requirement Description	Priority	Acceptance Criteria
FR-01	The system shall allow a new user to register by providing a unique email address, a full name, and a secure password. Registration must include email format validation and check for duplicate accounts.	High	Registration succeeds with valid unique email; fails with duplicate or invalid email
FR-02	The system shall allow a registered user to securely log in using their email and password. It must hash and verify passwords using the bcrypt algorithm and establish a secure server-side session upon successful authentication.	High	Valid credentials grant access; invalid credentials produce generic error message
FR-03	The system shall allow an authenticated user to add a new income transaction by specifying date, amount, selecting a category from a dropdown, and adding an optional note. The user shall also be able to view a paginated history of all their income transactions.	High	Income records persist accurately; history displays in correct chronological order
FR-04	The system shall allow an authenticated user to add a new expense transaction by specifying date, amount (in ETB), selecting a category from a dropdown, and adding an optional note. The user shall also be able to view a paginated history of all their expense transactions.	High	Expense records persist accurately; budget warnings trigger correctly
FR-05	The system shall allow an authenticated user to create, edit, and delete custom categories. Each category must be	Medium	Categories maintain type integrity; deletion preserves historical transaction data

	designated as either an "Income" or "Expense" type. Deleting a category used in existing transactions shall reassign those transactions to a default "Uncategorized" label, not delete the transactions.		
FR-06	The system shall allow an authenticated user to set a monthly budget limit (in ETB) for any specific expense category for a chosen future month.	High	Budget amounts save correctly; duplicate month-category combinations are prevented
FR-07	The system shall visually compare the user's set budget against the actual spending for a selected category and month. This shall be displayed using a progress bar and numeric values. A clear visual warning must be displayed if actual spending exceeds the budget.	High	Visual indicators update in real-time; warnings trigger at appropriate thresholds
FR-08	The system shall generate graphical financial reports based on user data. At a minimum, reports shall include weekly spending charts, monthly income vs. expense trend lines, and a yearly summary.	Medium	Charts render accurately; data aggregation produces correct results
FR-09	The system shall allow an authenticated user to export a summary report for the **current month** as a professionally formatted PDF document, including key totals and charts.	Medium	PDF generates with correct formatting; includes all specified data elements
FR-10	The system shall provide a secure logout function that terminates the user's server-side session, clears relevant client-side tokens, and redirects the user to the login page, preventing back-button access to protected pages.	High	Session invalidates completely; protected pages inaccessible after logout

3.2 Non-Functional Requirements

Non-Functional Requirements: Specifications describing how the system performs its functions, including quality attributes like performance, security, usability, reliability, and maintainability.

3.2.1 Usability Requirements

1. The system shall achieve a mean System Usability Scale score exceeding 70 points from a representative sample of at least 15 first-time users completing standardized evaluation protocols.
2. Interface design must follow Nielsen's 10 usability heuristics with particular emphasis on consistency across modules, error prevention through validation, and recognition rather than recall for navigation elements.
3. Task completion metrics shall demonstrate that 85% of test users successfully execute core operations including transaction entry, budget setting, and report generation without external assistance during initial attempts.
4. Learnability measurements shall indicate average time to first successful transaction recording under five minutes for users with basic computer literacy.

3.2.2 Security Requirements

1. The system shall maintain zero critical or high-severity security vulnerabilities as assessed through comprehensive testing including manual code review and automated scanning aligned with OWASP Top 10 guidelines.
2. Password storage implementation must utilize bcrypt adaptive hashing algorithm with work factor 12 ensuring computational complexity against brute-force attacks.
3. All database interactions shall employ parameterized queries or prepared statements eliminating SQL injection vulnerabilities through query-data separation.
4. User input validation shall implement both client-side feedback for user experience and server-side enforcement for security, with all dynamic output encoding applying appropriate context-specific escaping to prevent cross-site scripting attacks.

3.2.3 Performance Requirements

1. The system shall maintain average page load times under two seconds for authenticated dashboard views and transaction listings when accessing datasets containing up to 1000 transactions over standard broadband connections.
2. Transaction processing operations including income and expense recording shall complete confirmation within one second of form submission under normal system load conditions.
3. Report generation with graphical visualizations shall render within three seconds for monthly datasets, while PDF export operations shall complete document generation within five seconds for standard financial summaries. These performance metrics shall remain consistent with concurrent user loads simulating up to 10 simultaneous active sessions.

3.2.4 Reliability Requirements

1. The system shall ensure 99.9% data persistence integrity where no user-entered transactions are lost under normal operating conditions including graceful shutdown procedures and standard browser closure scenarios.
2. Error recovery mechanisms shall provide user-friendly notification messages and retry pathways within 30 seconds of recoverable failure conditions such as database connection timeouts or temporary service interruptions.
3. System availability targets shall maintain 99.5% uptime excluding scheduled maintenance periods, with automated monitoring detecting service degradation and triggering appropriate alert responses.

3.2.5 Maintainability Requirements

1. The source code shall implement modular architecture with clear separation between presentation, business logic, and data access layers following established software design principles.
2. Each major functional component shall include comprehensive inline documentation explaining purpose, parameters, return values, and exceptional conditions.
3. The codebase shall support future enhancements through extensible design patterns and configuration externalization allowing behavior modification without source code changes.
4. Development shall produce accompanying technical documentation including architecture diagrams, database schema specifications, and deployment procedures facilitating future maintenance activities.

3.3 Other Requirements

3.3.1 Business Requirements

- The system shall provide a completely free alternative to subscription-based budgeting tools
- The system shall ensure complete user data privacy through local storage implementation. It shall avoid cloud synchronization and third-party data sharing.
- The system shall promote financial literacy among diverse socioeconomic groups. It shall provide educational tools for better money management.
- The system shall function without persistent internet connectivity. It shall provide full offline capabilities for users with limited access.
- The system shall demonstrate academic mastery of software engineering principles.
- The system shall establish a foundation for potential future commercial development. It shall use extensible architecture for enhancements.

3.3.2 User Requirements

- Users shall be able to track daily income and expenses with minimal effort.
- Users shall be able to set and monitor monthly budgets for different categories.

- Users shall be able to view financial trends through visual reports and charts.
- Users shall be able to organize transactions using customizable categories.
- Users shall be able to access financial data securely without internet dependency.
- Users shall be able to export financial summaries for external documentation.
- Users shall receive warnings when approaching or exceeding budget limits.
- Users shall maintain complete control over personal financial data.

3.3.3 Legal and Regulatory Requirements

The system development and operation adhere to fundamental privacy principles emphasizing user data sovereignty and minimal data collection. Implementation follows privacy-by-design architecture where data processing occurs locally without external transmission unless explicitly requested through export functions. Licensing arrangements utilize open-source models allowing free usage, modification, and distribution with attribution requirements protecting intellectual property while encouraging community enhancement.

Regulatory compliance focuses on data protection principles including purpose limitation, data minimization, and storage limitation without specific jurisdictional mandates due to local deployment nature.

3.3.4 Documentation Requirements

User Documentation: Comprehensive user manual provides step-by-step guidance for system utilization beginning with installation procedures for various environments, initial configuration steps for personalization, daily operation instructions for all features, troubleshooting guides for common issues, and advanced usage techniques for power users. Documentation organization follows task-oriented structure with clear progression from basic to advanced functionality, supported by visual examples and practical scenarios illustrating real-world application.

Technical Documentation: Complete technical specification includes architecture overview describing component relationships, API documentation detailing endpoint specifications and data formats, database schema documentation with entity-relationship diagrams and data dictionaries, deployment guides covering various environment configurations, and maintenance procedures for routine operations and troubleshooting. Technical documentation targets system administrators and developers with appropriate detail level for implementation and support activities.

3.3.5 Training Requirements

Formal training requirements remain minimal due to intuitive interface design and comprehensive embedded assistance. Contextual help systems provide immediate guidance during user interactions, while progressive disclosure introduces complexity gradually as users advance through functionality.

The user manual serves as self-paced learning resource with structured modules covering specific competency areas. Optional training materials include video tutorials demonstrating common workflows, quick reference guides summarizing frequent tasks, and example scenarios illustrating

practical application. Training approach emphasizes discovery learning through safe exploration with protective measures preventing irreversible data modifications during initial learning phases.

3.3.6 Quality Requirements

Quality assurance implements multi-layered approach throughout development lifecycle with specific metrics for evaluation. Code quality assessment includes static analysis for syntax validation and pattern compliance, unit test coverage exceeding 90% for critical paths, and peer review processes ensuring consistency and best practice adherence.

System quality verification incorporates functional testing against requirement specifications, integration testing for component interactions, performance testing under simulated load conditions, and security testing through vulnerability scanning and penetration attempts. Process quality employs continuous integration with automated build verification, regular code quality metrics collection and analysis, and iterative improvement cycles based on measurement outcomes and user feedback.

Chapter 4: Design Analysis

4.1 Use Case Modeling

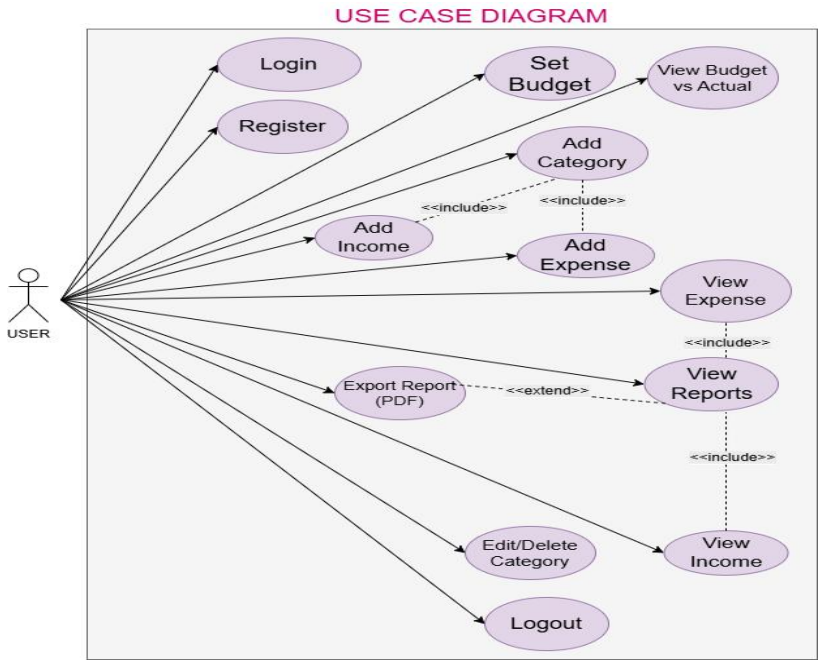


Figure 4.1: Use Case Diagram

Description: The **Use Case Diagram** provides high-level visualization of interactions between the primary User actor and system functionalities.

Relationship notations include ***«include»*** dependencies indicating mandatory inclusions (such as Add Category within Add Income/Expense flows) and ***«extend»*** relationships showing optional extensions (particularly Export PDF extending View Reports).

The **system boundary** clearly delineates functionality scope, while actor representation establishes single-user focus without administrative roles. This diagram serves as the foundational visual agreement on system behavior before detailed design specification.

4.1.1 Use Case Description

Table 4.1 UC-01: Register User

Attribute	Description
Use Case ID	UC-01
Use Case Name	Register User
Primary Actor	New User
Secondary Actors	System, Database
Description	A new user creates an account in the system by providing personal information including name, email, and password.
Preconditions	1. User is not logged in 2. User has access to registration page 3. System is operational
Postconditions	1. User account is created in database 2. User can now log in with created credentials 3. Confirmation message displayed
Basic Flow	1. User navigates to registration page

	2. System displays registration form 3. User enters name, email, password, confirm password 4. System validates input 5. System checks email uniqueness 6. System creates account with hashed password 7. System redirects to login page with success message
Alternative Flows	**A1: Duplicate Email** 3a. User enters existing email 4a. System detects duplicate 5a. System displays error message 6a. User corrects email or uses different one **A2: Invalid Password** 3b. User enters weak password 4b. System validates password strength 5b. System displays password requirements 6b. User enters stronger password
Priority	High
Special Requirements	Password must be hashed using bcrypt before storage

Table 4.2 UC-02: Login User

Attribute	Description
Use Case ID	UC-02
Use Case Name	Login User
Primary Actor	Registered User
Secondary Actors	System, Database, Session Manager
Description	Registered user authenticates with email and password to access the system.
Preconditions	1. User has a registered account 2. User is not currently logged in 3. System is operational
Postconditions	1. User session is established 2. User is redirected to dashboard 3. Session timer starts
Basic Flow	1. User navigates to login page 2. System displays login form 3. User enters email and password 4. System validates credentials 5. System creates session 6. System redirects to dashboard
Alternative Flows	**A1: Invalid Credentials** 3a. User enters wrong password 4a. System validates and finds mismatch 5a. System displays generic error 6a. User re-enters credentials **A2: Remember Me Option** 3b. User checks "Remember me" 6b. System sets persistent cookie 7b. Session lasts longer duration
Priority	High
Special Requirements	Password verification must use bcrypt compare function

Table 4.3 UC-03: Add Income

Attribute	Description
Use Case ID	UC-03
Use Case Name	Add Income
Primary Actor	Authenticated User
Secondary Actors	System, Database, Category Manager
Description	User records an income transaction with details including amount, date, category, and optional notes.
Preconditions	1. User is logged in 2. Income categories exist or can be created 3. User has permission to add transactions
Postconditions	1. Income transaction saved to database

	2. User balance updated 3. Recent transactions list updated
Basic Flow	1. User navigates to Income page 2. System displays income form and recent transactions 3. User enters amount, date, selects category, adds note 4. System validates input 5. System saves transaction 6. System updates dashboard balance 7. System displays success message
Alternative Flows	**A1: Create New Category** 3a. Category dropdown doesn't have needed category 4a. User clicks "Add New Category" 5a. System shows category creation modal 6a. User creates new category 7a. System returns to income form with new category selected **A2: Quick Add from Dashboard** 1a. User clicks "Quick Add Income" from dashboard 2a. System shows simplified income form 3a. User enters only amount and category
Priority	High
Special Requirements	Transaction should update real-time balance calculation

Table 4.4 UC-04: Add Expense

Attribute	Description
Use Case ID	UC-04
Use Case Name	Add Expense
Primary Actor	Authenticated User
Secondary Actors	System, Database, Budget Manager, Category Manager
Description	User records an expense transaction with budget checking and category assignment.
Preconditions	1. User is logged in 2. Expense categories exist or can be created 3. Budget may or may not be set for category
Postconditions	1. Expense transaction saved 2. Budget progress updated if applicable 3. Balance updated
Basic Flow	1. User navigates to Expense page 2. System displays expense form 3. User enters amount, date, selects category, adds note 4. System validates and checks budget 5. System saves expense 6. System updates budget progress 7. System displays success message
Alternative Flows	**A1: Budget Warning** 4a. Expense exceeds category budget 5a. System shows warning message 6a. User confirms to proceed 7a. System saves with warning flag **A2: Recurring Expense** 3b. User marks as recurring 4b. System prompts for frequency 5b. System schedules future expenses
Priority	High
Special Requirements	Must check against budget limits if set

Table 4.5 UC-05: Add Category

Attribute	Description
Use Case ID	UC-05
Use Case Name	Add Category
Primary Actor	Authenticated User
Secondary Actors	System, Database

Description	User creates a new custom category for classifying income or expenses.
Preconditions	1. User is logged in 2. User is on category management or transaction form
Postconditions	1. New category added to user's category list 2. Category available for transaction classification
Basic Flow	1. User navigates to Category Management or clicks "Add Category" from transaction form 2. System displays category creation form 3. User enters category name and selects type (Income/Expense) 4. System validates uniqueness 5. System saves category 6. Category appears in dropdown lists
Alternative Flows	**A1: From Transaction Form** 1a. User is adding transaction 2a. Clicks "New Category" in dropdown 3a. System shows inline category creation 4a. After creation, returns to transaction form with new category selected **A2: Bulk Category Creation** 3b. User imports categories from CSV 4b. System validates and creates multiple categories
Priority	Medium
Special Requirements	Categories must be user-specific (not global)

Table 4.6 UC-06: Edit/Delete Category

Attribute	Description
Use Case ID	UC-06
Use Case Name	Edit/Delete Category
Primary Actor	Authenticated User
Secondary Actors	System, Database, Transaction Manager
Description	User modifies existing category details or removes a category with proper handling of associated transactions.
Preconditions	1. User is logged in 2. Category exists 3. User is on category management page
Postconditions	1. Category updated or marked as deleted 2. Associated transactions reassigned if deleted 3. Category lists updated
Basic Flow	**Edit Flow:** 1. User selects category to edit 2. System shows edit form with current values 3. User modifies name/type 4. System validates changes 5. System updates category 6. System shows success message **Delete Flow:** 1. User selects category to delete 2. System shows confirmation with transaction count 3. User confirms deletion 4. System reassigns transactions to "Uncategorized" 5. System marks category as deleted 6. System shows deletion confirmation
Alternative Flows	**A1: Cancel Deletion** 3a. User cancels deletion 4a. System returns to category list **A2: Edit with Transactions** 4a. Category has transactions 5a. System warns about affecting historical data

	6a. User confirms to proceed
Priority	Medium
Special Requirements	Soft delete implementation with transaction protection

Table 4.7 UC-07: Set Budget

Attribute	Description
Use Case ID	UC-07
Use Case Name	Set Budget
Primary Actor	Authenticated User
Secondary Actors	System, Database, Category Manager
Description	User establishes monthly spending limits for specific expense categories.
Preconditions	1. User is logged in 2. Expense categories exist 3. User is on budget planning page
Postconditions	1. Budget saved for category-month combination 2. Budget monitoring activated 3. Progress tracking begins
Basic Flow	1. User navigates to Budget Planning page 2. System shows budget form and existing budgets 3. User selects month (future or current) 4. User selects expense category 5. User enters budget amount 6. System validates input 7. System saves budget 8. System shows budget in list with progress
Alternative Flows	**A1: Edit Existing Budget** 3a. Budget already exists for category-month 4a. System shows current amount in form 5a. User modifies amount 7a. System updates existing budget **A2: Copy Previous Month Budget** 3b. User selects "Copy from previous month" 4b. System auto-fills with last month's budgets 5b. User adjusts as needed
Priority	High
Special Requirements	Budget should be editable until month ends

Table 4.8 UC-08: View Budget Progress

Attribute	Description
Use Case ID	UC-08
Use Case Name	View Budget Progress
Primary Actor	Authenticated User
Secondary Actors	System, Database, Expense Aggregator
Description	User monitors spending against budget with visual progress indicators and warnings.
Preconditions	1. User is logged in 2. Budgets are set for some categories 3. Expenses may or may not be recorded
Postconditions	1. Budget progress displayed visually 2. Warnings shown if thresholds exceeded 3. User aware of spending status
Basic Flow	1. User navigates to Budget page 2. System retrieves budgets and expenses for current month 3. System calculates spending percentages 4. System displays progress bars with colors 5. System shows numerical comparison (budget vs actual) 6. System highlights categories needing attention
Alternative Flows	**A1: Select Different Month**

	2a. User selects different month from dropdown 3a. System retrieves data for selected month 4a. System displays historical progress **A2: Drill Down to Details** 5a. User clicks on a category progress bar 6a. System shows detailed transaction list for that category-month
Priority	High
Special Requirements	Real-time updates when expenses added

Table 4.9 UC-09: View Reports

Attribute	Description
Use Case ID	UC-09
Use Case Name	View Reports
Primary Actor	Authenticated User
Secondary Actors	System, Database, Chart Generator
Description	User generates and views graphical financial reports with various timeframes and visualizations.
Preconditions	1. User is logged in 2. Financial data exists (transactions recorded) 3. System has charting capability
Postconditions	1. Reports displayed with charts 2. User gains financial insights 3. Export options available
Basic Flow	1. User navigates to Reports page 2. System shows timeframe selector (Week/Month/Year) 3. User selects timeframe 4. System aggregates transaction data 5. System generates charts (pie, bar, line) 6. System displays charts with summaries 7. User interacts with charts (hover, click)
Alternative Flows	**A1: Custom Date Range** 3a. User selects "Custom Range" 4a. System shows date picker 5a. User selects start and end dates 6a. System generates report for custom period **A2: Category Filtering** 5a. User selects specific categories to include 6a. System regenerates charts with filtered data
Priority	Medium
Special Requirements	Charts must be responsive and accessible

Table 4.10 UC-10: Export PDF Report

Attribute	Description
Use Case ID	UC-10
Use Case Name	Export PDF Report
Primary Actor	Authenticated User
Secondary Actors	System, PDF Generator, Chart Renderer
Description	User exports current financial report as a formatted PDF document for sharing or archiving.
Preconditions	1. User is logged in 2. Report is currently being viewed 3. PDF generation library available
Postconditions	1. PDF file generated 2. Download initiated to user's device

	3. Report preserved in document format
Basic Flow	1. User is viewing reports 2. User clicks "Export PDF" button 3. System shows generation progress 4. System converts charts to images 5. System formats data into PDF template 6. System generates PDF file 7. Browser downloads PDF file 8. System shows download confirmation
Alternative Flows	**A1: Scheduled Export** 2a. User sets up monthly auto-export 3a. System emails PDF at month end **A2: Custom PDF Content** 4a. User selects specific sections to include 5a. System generates customized PDF
Priority	Medium
Special Requirements	Charts must render as high-quality images in PDF

Table 4.11 UC-11: Logout

Attribute	Description
Use Case ID	UC-11
Use Case Name	Logout
Primary Actor	Authenticated User
Secondary Actors	System, Session Manager
Description	User securely terminates their session and exits the application.
Preconditions	1. User is logged in 2. Session is active 3. User is on any authenticated page
Postconditions	1. Session terminated 2. All tokens cleared 3. User redirected to login page 4. Protected content inaccessible
Basic Flow	1. User clicks "Logout" link/button 2. System confirms logout intention 3. User confirms logout 4. System destroys server session 5. System clears client-side tokens/cookies 6. System redirects to login page 7. System displays "Successfully logged out" message
Alternative Flows	**A1: Auto-Logout** 1a. Session times out due to inactivity 2a. System automatically performs logout 3a. System shows "Session expired" message on login page **A2: Logout All Devices** 3a. User selects "Logout from all devices" 4a. System invalidates all user sessions
Priority	High
Special Requirements	Must prevent session fixation attacks

Table 4.12 UC-12: View Dashboard

Attribute	Description
Use Case ID	UC-12
Use Case Name	View Dashboard
Primary Actor	Authenticated User
Secondary Actors	System, Database, Aggregator
Description	User views the main dashboard with financial overview, recent activity, and quick actions.
Preconditions	1. User is logged in 2. User has financial data (may be empty) 3. Dashboard module is available
Postconditions	1. Dashboard displayed with current data

	2. User sees financial summary 3. Quick access to common actions
Basic Flow	1. User logs in or navigates to dashboard 2. System retrieves current month data 3. System calculates totals: income, expense, balance 4. System retrieves recent transactions (last 10) 5. System checks budget status for warnings 6. System displays dashboard with all components 7. User interacts with dashboard elements
Alternative Flows	**A1: First-Time User Dashboard** 2a. No transactions exist yet 3a. System shows welcome tutorial 4a. System suggests first actions (add category, set budget) **A2: Dashboard Widget Customization** 6a. User rearranges dashboard widgets 7a. System saves layout preferences
Priority	High
Special Requirements	Dashboard must load within 2 seconds

4.2 Structural Design

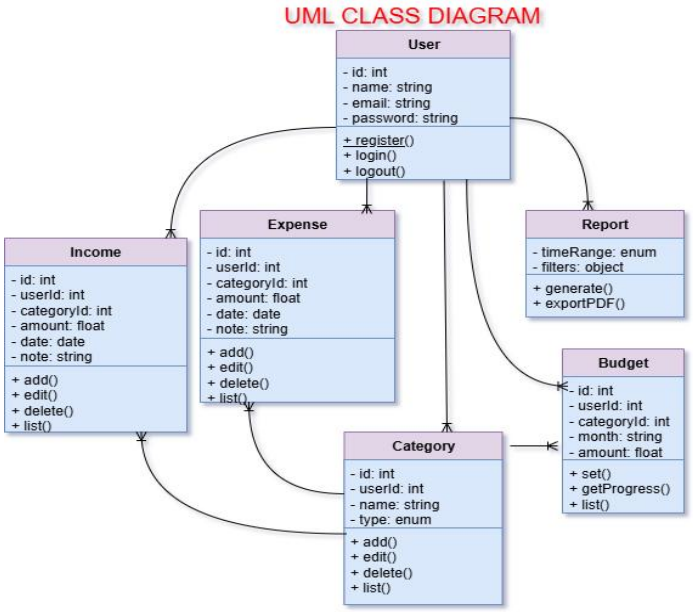


Figure 4.2: Class Diagram

Description: The **Class Diagram** defines the system's static architecture through six primary classes with their attributes, methods, and relationships.

The **User** class serves as the central entity with authentication methods and associations to all financial data classes.

Category class enables user-specific classification with type enforcement through enumeration constraints.

Income and **Expense** classes mirror transaction structures with foreign key relationships to both User and Category entities.

Budget class implements planning functionality with composite key constraints ensuring single budget per category-month combination.

Report class operates as a controller entity coordinating data aggregation and presentation without direct database persistence.

Relationship cardinalities clearly indicate one-to-many associations between User and transactional classes, while many-to-one relationships connect transactions to Category instances. This diagram directly informs database schema design and object-oriented implementation structure.

4.3 Behavioral Design

4.3.1 Sequence Diagrams

Sequence Diagrams: UML diagrams that show how objects interact in a particular sequence of time, illustrating the flow of messages between objects to accomplish a specific functionality.

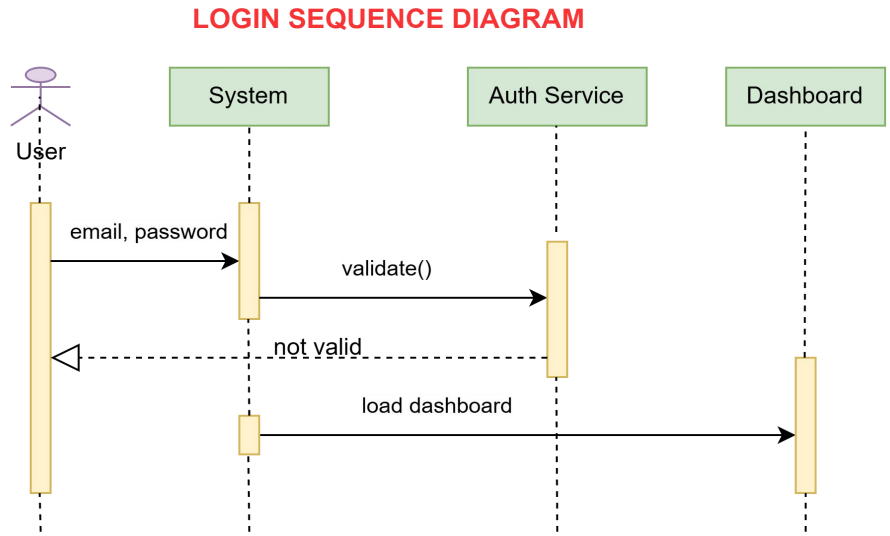


Figure 4.3: Login Sequence Diagram

Login Sequence: Illustrates the secure authentication workflow beginning with credential submission through the Login Interface, passing through Authentication Controller validation, User object password verification against database-stored bcrypt hashes, session establishment upon successful authentication, and dashboard loading with personalized data retrieval. The sequence emphasizes security considerations including hash comparison rather than password decryption and session regeneration preventing fixation attacks.

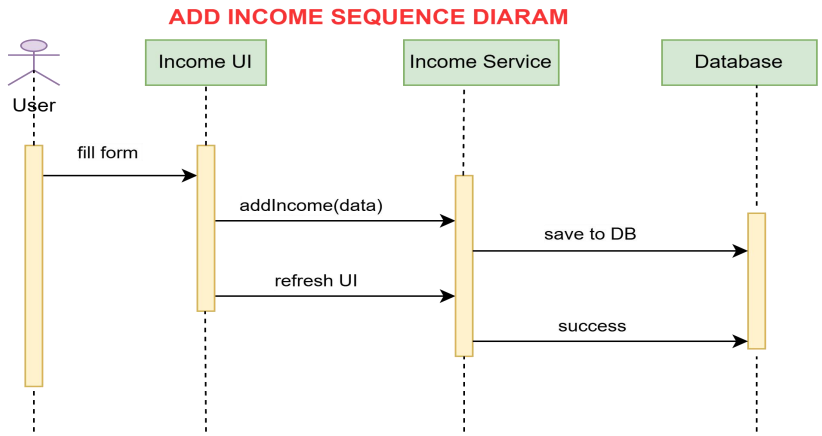


Figure 4.4: Add Income Sequence Diagram

Add Income Sequence: Demonstrates transaction creation starting with form submission from Income Interface, server-side validation by Transaction Controller, Income object instantiation with data encapsulation, database insertion through parameterized queries, success confirmation propagation back through controller layers, and interface updating with refreshed transaction lists. The sequence highlights data validation separation between client-side user experience and server-side security enforcement.

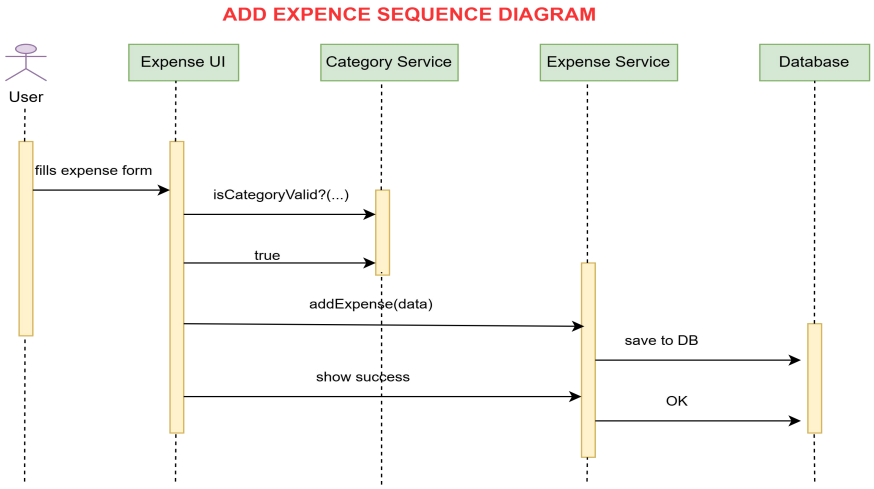


Figure 4.5: Add Expense Sequence Diagram

Add Expense Sequence: Parallels income addition with expense-specific considerations including budget validation checks and category type restrictions. The sequence shows identical structural patterns with Expense object substitution, emphasizing code reusability and consistent design patterns across similar functionalities while maintaining logical separation.

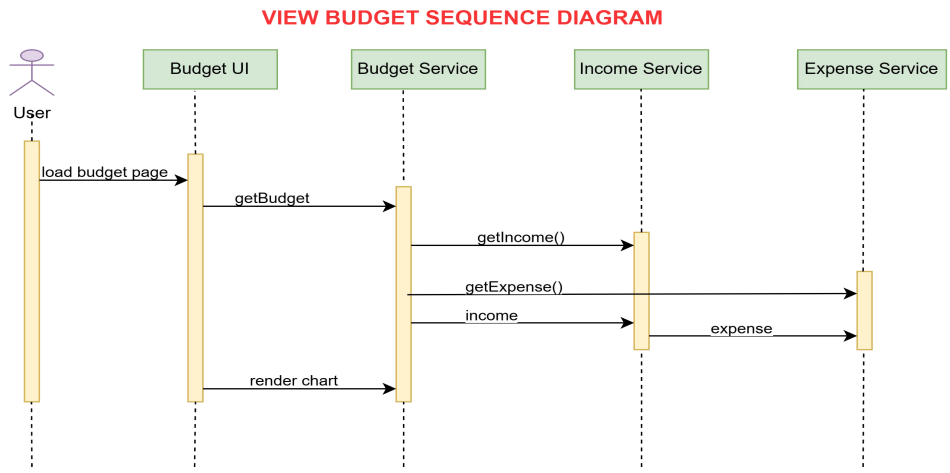


Figure 4.6: View Budget vs Actual Sequence Diagram

Budget Comparison Sequence: Models the budgeting logic flow involving month and category selection from Budget Interface, dual data retrieval for planned budget amounts and actual expense aggregates, percentage calculation and status determination, and visual presentation with conditional warning display. The sequence showcases data aggregation techniques and conditional interface responses based on business logic outcomes.

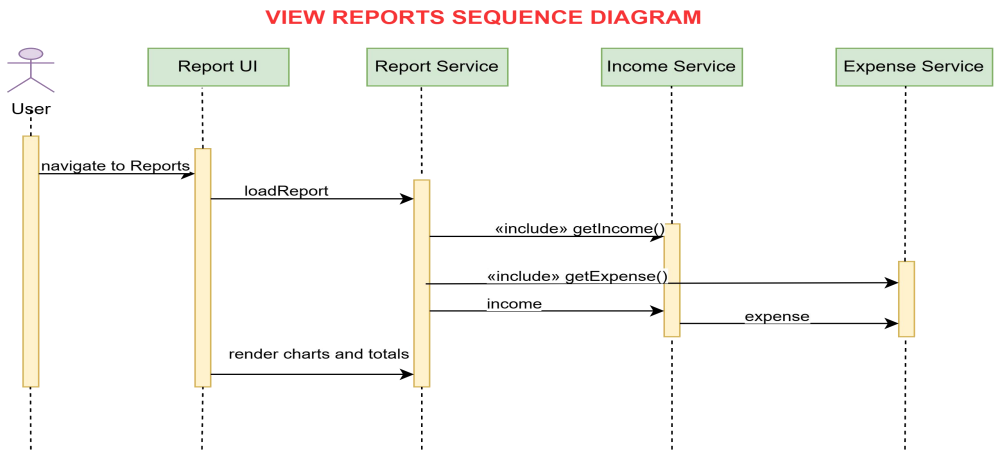


Figure 4.7: View Reports Sequence Diagram

PDF Export Sequence: Details the report generation and document creation process beginning with export request from Reports Interface, data aggregation through Report object queries, summary computation and chart data preparation, PDF library invocation with template processing, binary document generation, and HTTP response triggering browser download. The sequence illustrates complex multi-step processes involving external library integration and file handling operations.

4.3.2 Activity Diagrams

Activity Diagrams: UML diagrams that model workflow or business processes, showing the flow from one activity to another with decision points, parallel processing, and synchronization.

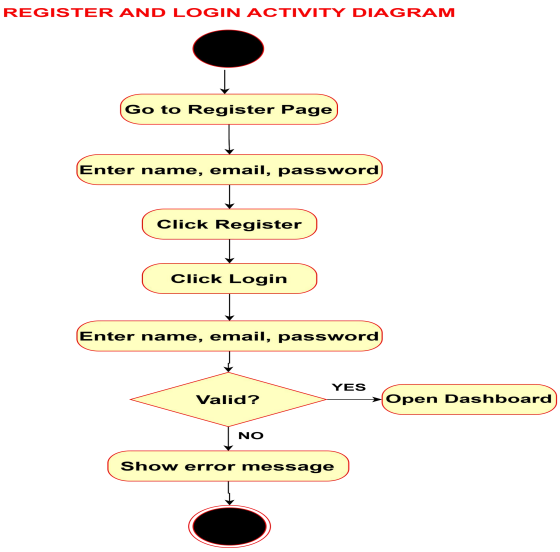


Figure 4.8: User Registration & Login Activity Diagram

Registration & Login: Models the entry point workflow with decision branching for new versus existing users, registration subprocess with validation steps, login authentication with success/failure paths, and dashboard access as the terminal state. The diagram clarifies user onboarding processes with error recovery loops and success progression.

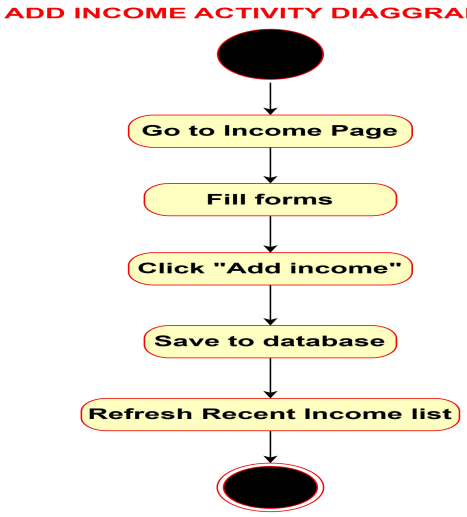


Figure 4.9: Add Income Activity Diagram

Transaction Addition: Details the income recording process from interface navigation through form completion, validation checks, database persistence, interface updating, and confirmation display. The workflow emphasizes validation failure handling with user correction pathways and successful flow completion.

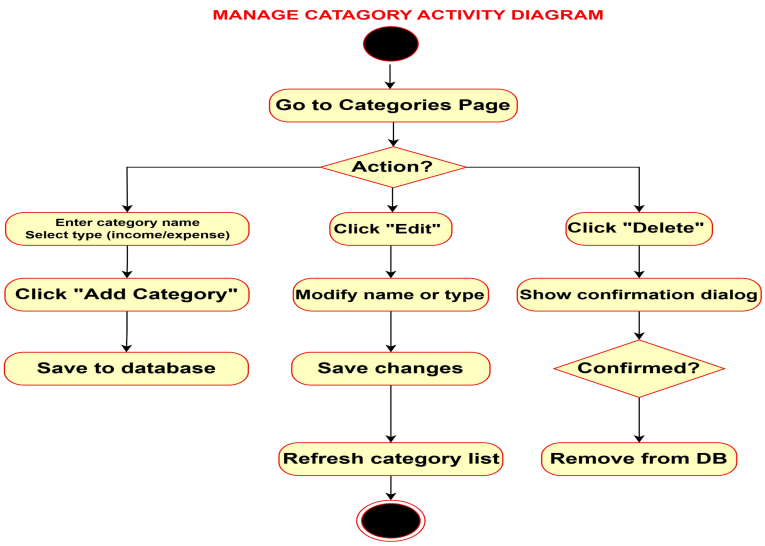


Figure 4.10: Manage Categories Activity Diagram

Category Management: Illustrates category lifecycle operations including creation, modification, and deletion with specialized handling for category deletion involving transaction reassignment confirmation. The activity flow demonstrates protective measures preserving data integrity during structural changes.

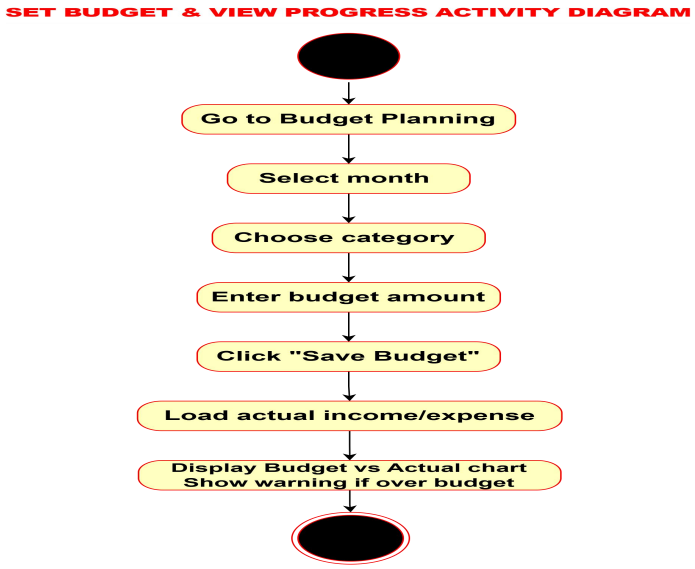


Figure 4.11: Set Budget & View Progress Activity Diagram

Budget Management: Combines budget setting and monitoring activities showing parameter selection, amount specification, saving operations, automatic comparison triggering, and conditional warning display based on exceedance calculations. The diagram visualizes the interactive feedback loop central to budget tracking functionality.

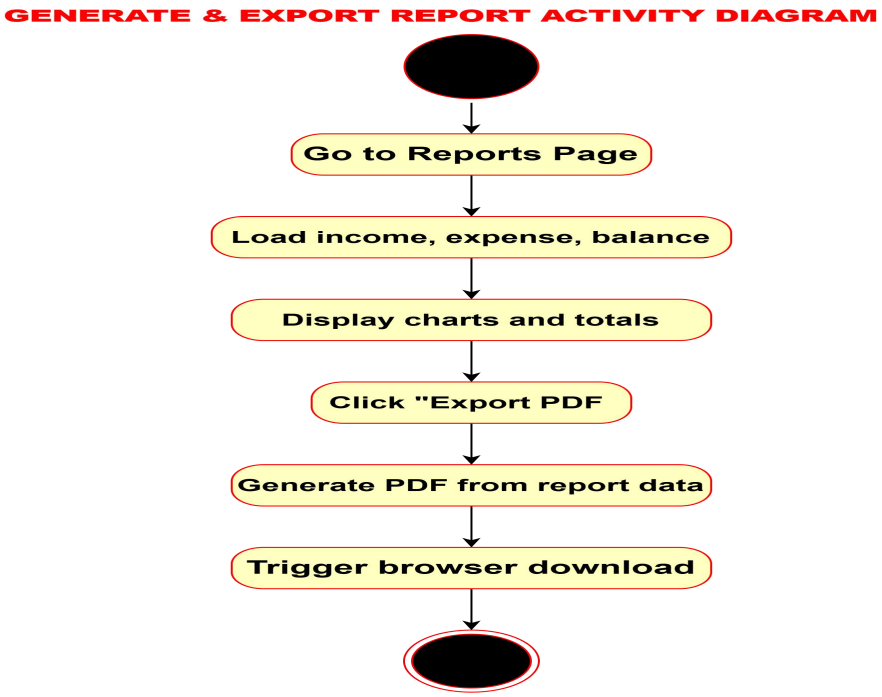


Figure 4.12: Generate & Export Report Activity Diagram

Report Generation: Models the reporting process from interface loading with automatic data retrieval through optional export initiation, PDF generation, and download completion. The workflow distinguishes between viewing and exporting activities while maintaining data consistency.

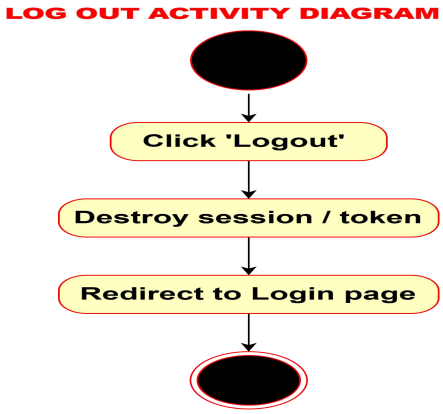


Figure 4.13: Logout Activity Diagram

Logout Process: Details secure session termination beginning with logout request, session destruction, token clearance, login redirection, and confirmation messaging. The activity emphasizes security considerations with complete access revocation.

4.4 Traceability Matrix

Traceability Matrix: A document that maps and traces user requirements with system requirements, ensuring completeness and demonstrating that each requirement is addressed in design and testing.

Table 4.13: Requirements Traceability

Requirement ID	Use Case Reference	Sequence Diagram	Activity Diagram	Interface Component	Test Case Reference
FR-01, FR-02	UC-01, UC-02	Figure 4.3	Figure 4.8	Register Interface (Fig 5.1), Login Interface (Fig 5.2)	TC-AUTH-01 to TC-AUTH-08
FR-03	UC-03	Figure 4.4	Figure 4.9	Income Interface (Fig 5.4)	TC-TRAN-01 to TC-TRAN-06
FR-04	UC-04	Figure 4.5	Figure 4.9	Expense Interface (Fig 5.5)	TC-TRAN-07 to TC-TRAN-12
FR-05	UC-05, UC-06	Implied in Fig 4.4-4.5	Figure 4.10	Category Interface (Fig 5.6)	TC-CAT-01 to TC-CAT-09
FR-06, FR-07	UC-07, UC-08	Figure 4.6	Figure 4.11	Budget Interface (Fig 5.8)	TC-BUD-01 to TC-BUD-10
FR-08, FR-09	UC-09, UC-10	Figure 4.7	Figure 4.12	Reports Interface (Fig 5.7)	TC-REP-01 to TC-REP-08
FR-10	UC-11	-	Figure 4.13	Logout Component	TC-AUTH-09
NFR-01 to NFR-05	UC-01 to UC-12	Various	Various	All Interfaces	TC-NFR-01 to TC-NFR-15

-FR: Functional Requirement **-NFR**: Non-Functional Requirement **-UC**: Use Case **-TC**: Test Case **-Fig**: Figure

Chapter 5: External Interface Requirements

External Interface Requirements: Specifications defining how the system interacts with external entities including users, hardware, software, and communication protocols.

5.1 User Interfaces

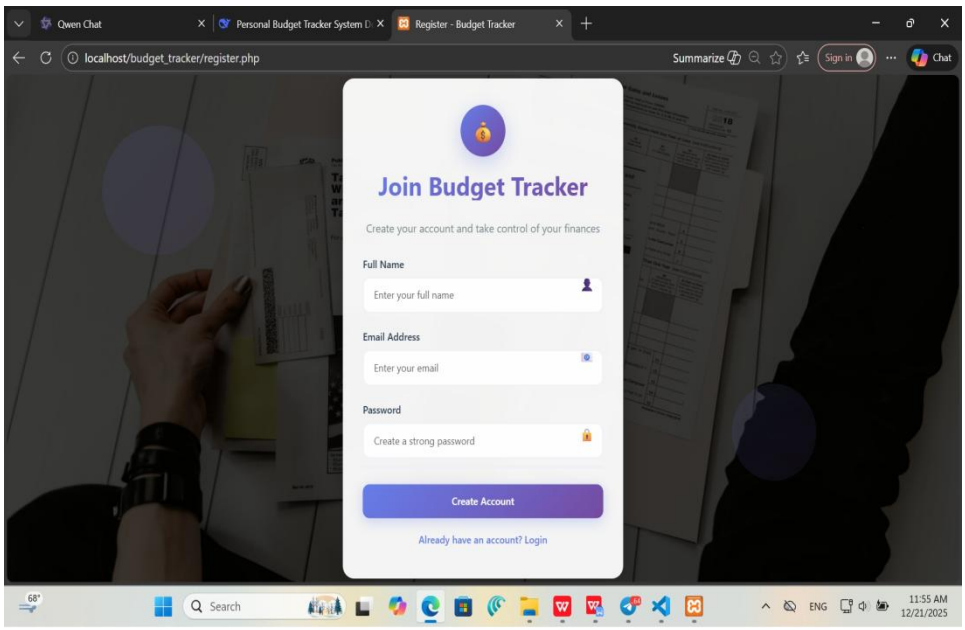


Figure 5.1: Register Interface

Description: The registration interface presents a clean, centered form with distinct sections for personal information including full name, email address, password with strength indicator, and password confirmation. Visual design employs minimalistic aesthetics with clear typographic hierarchy, inline validation providing immediate feedback, and accessible form controls. The interface implements progressive disclosure revealing password requirements during interaction, while submission controls provide clear call-to-action with loading states during processing. Error presentation utilizes contextual positioning adjacent to problematic fields with specific guidance for correction.

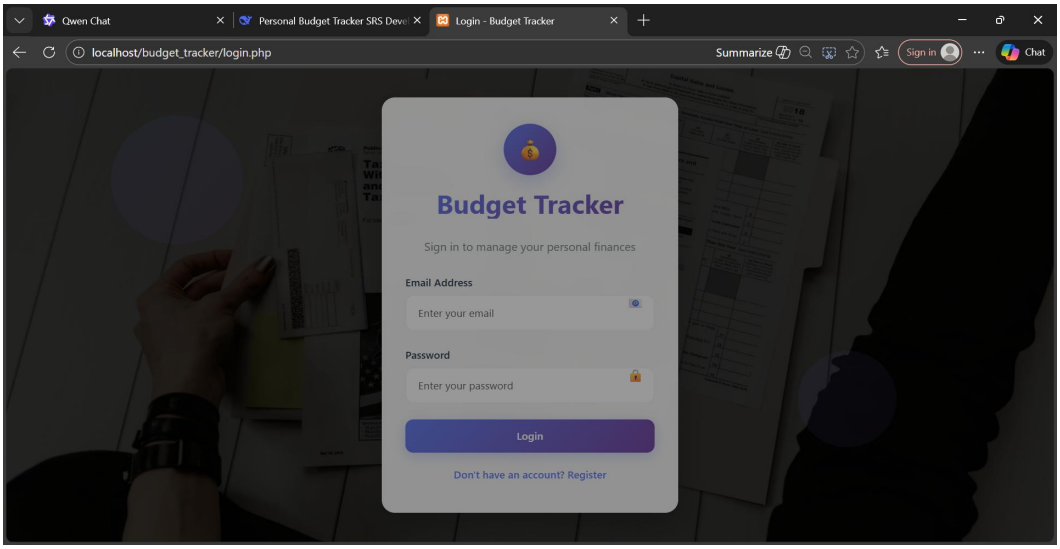


Figure 5.2: Login Interface

Description: The login interface maintains consistent styling with registration while focusing on authentication efficiency. The form includes email and password fields with persistent entry options for trusted devices, password visibility toggle for verification, and recovery options for credential issues. Security considerations influence design with generic error messaging preventing user enumeration, while successful authentication provides smooth transition to the dashboard. The interface balances security requirements with usability through remembered email addresses and clear session status indications.

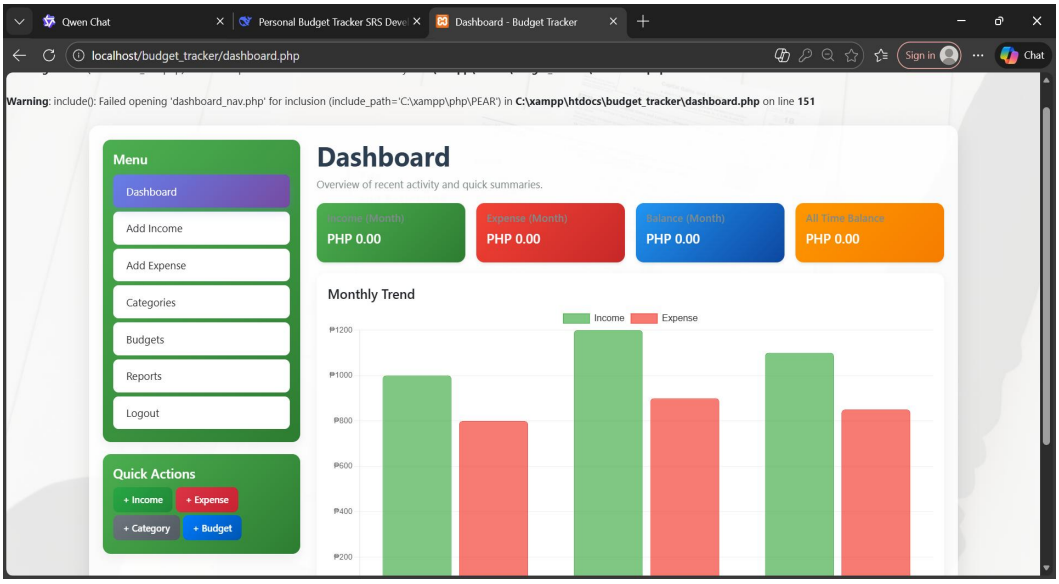


Figure 5.3: Dashboard Interface

Description: The dashboard serves as the application hub presenting key financial metrics through prominent summary cards displaying current month income, expenses, and net balance with comparative indicators. Visual design employs card-based layout with clear data hierarchy, color-coded financial categories, and trend indicators showing period-over-period changes. Interactive elements include quick-action buttons for common tasks, recent transaction listings with drill-down

capabilities, and budget status widgets highlighting attention-required categories. The interface maintains information density while ensuring scannability through deliberate whitespace and visual grouping.

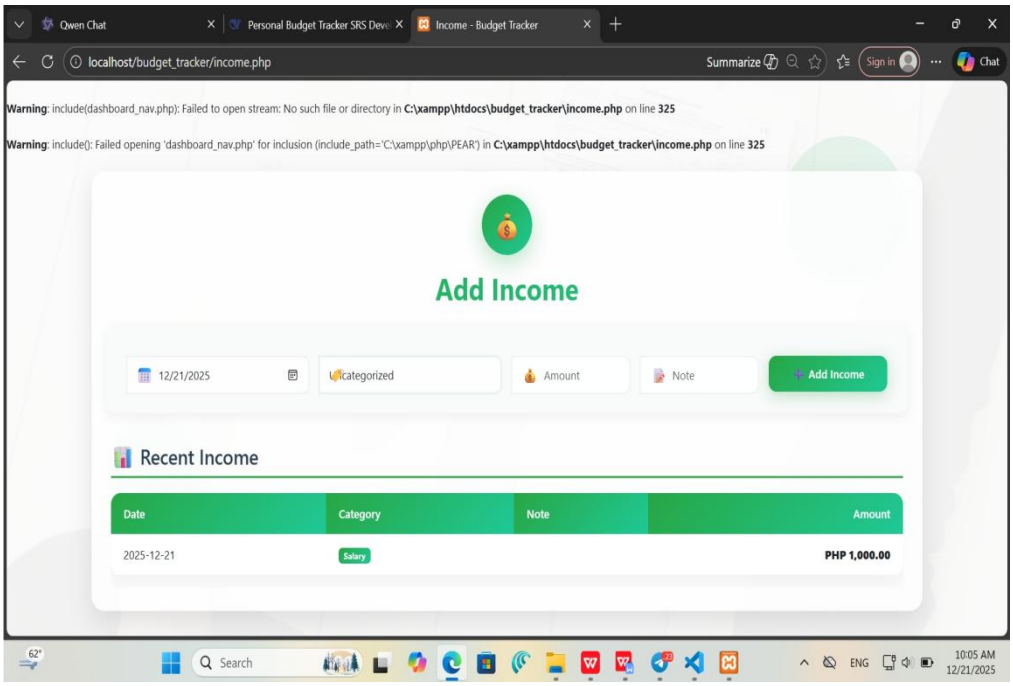


Figure 5.4: Add Income Interface

Description: The income management interface divides vertically between data entry form and transaction history listing. The entry section provides structured input for transaction date with calendar integration, amount with currency formatting, category selection from filtered dropdown, and optional notes with character counting. The history section displays paginated transactions with sortable columns, inline filtering controls, and action buttons for modification or deletion. Visual design emphasizes data integrity through validation states, while interaction design supports efficient data entry through sensible defaults and keyboard navigation.

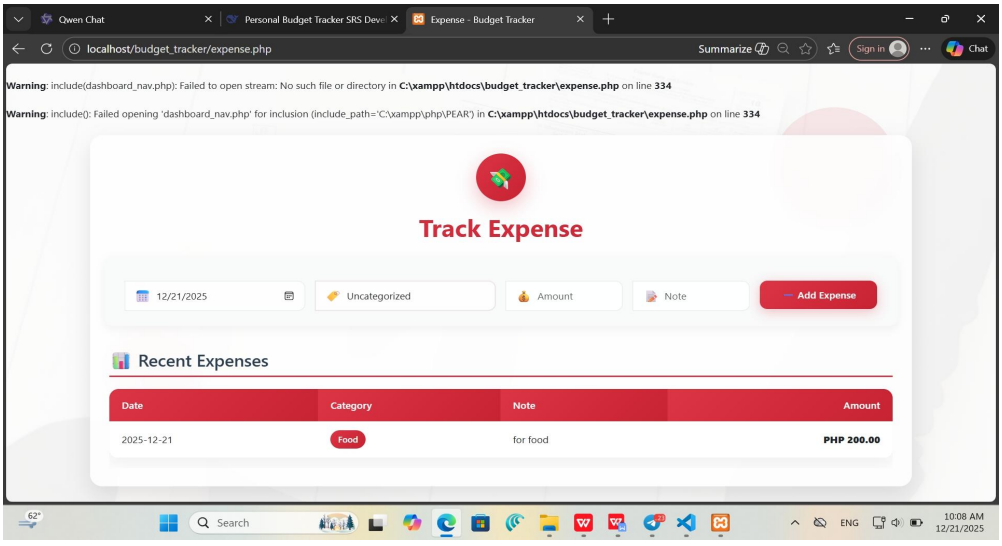


Figure 5.5:Add Expense Interface

Description: The expense interface shares structural similarity with income management while incorporating budget-aware features. The entry form includes additional validation checking category budget status with warning previews, while the history listing emphasizes budget impact through visual indicators showing transactions approaching or exceeding category limits. Design patterns maintain consistency with income interface for learnability while differentiating through color schemes and iconography appropriate to expense tracking context.

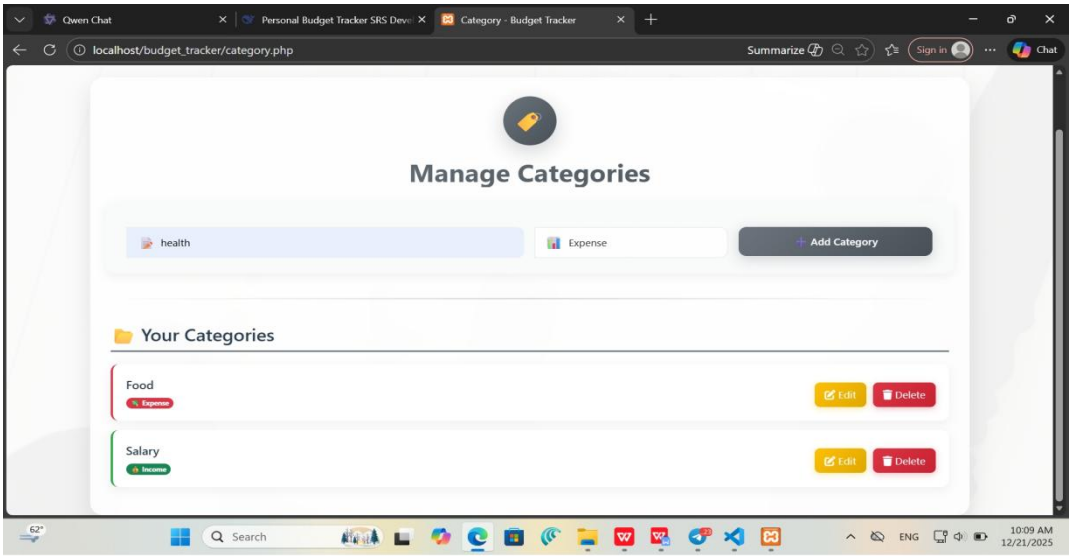


Figure 5.6: Category Interface

Description: The category management interface provides dual functionality for creation and administration of financial categories. The creation form includes name input with uniqueness validation and type selection through radio button groups. The administration panel displays categorized lists with usage statistics, modification controls, and deletion actions with protective confirmation dialogs explaining transaction reassignment consequences. Visual design distinguishes between income and expense categories through color coding and icon differentiation, while interaction design prevents accidental modifications through deliberate action requirements.

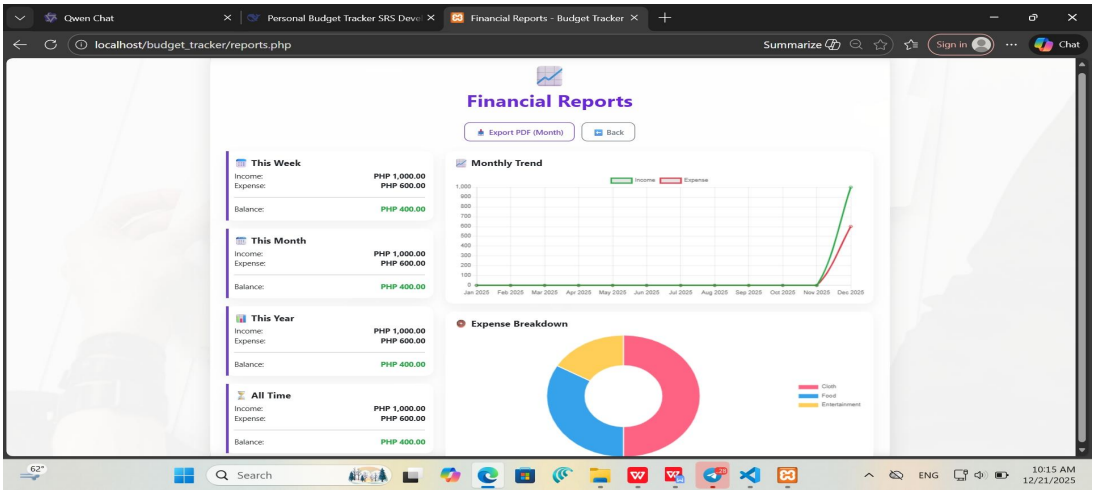


Figure 5.7: Reports Interface

Description: The reporting interface presents financial analytics through multiple coordinated visualization types. The control panel provides period selection with preset ranges and custom date specification, category filtering with multi-select capabilities, and chart type toggles for different analytical perspectives. Visualization area includes coordinated charts with drill-down capabilities, summary statistics with export-ready formatting, and comparison metrics showing period-over-period changes. The interface emphasizes analytical flexibility while maintaining performance through progressive loading and client-side rendering optimizations.

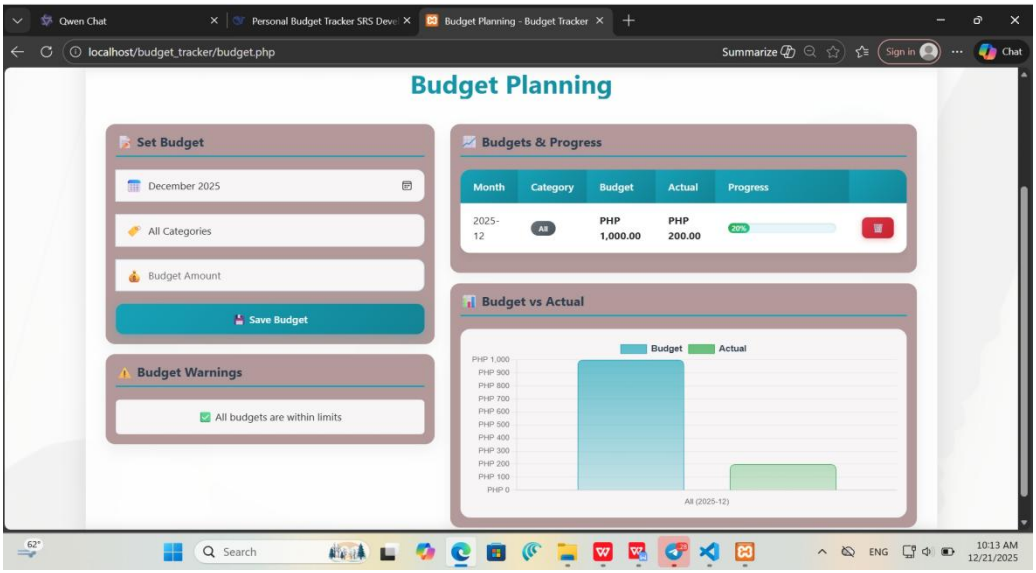


Figure 5.8: Budget Interface

Description: The budget interface combines planning controls with monitoring visualizations in a unified workspace. The planning section provides month selection, category assignment, and amount specification with historical reference displays showing previous period actuals. The monitoring section displays budget utilization through progress bars with color-coded status indicators, numerical comparisons showing planned versus actual amounts, and warning panels highlighting exceedance conditions. Interactive elements enable budget adjustment with revision tracking, while visualization components support both summary and detailed views of budget performance.

5.2 Hardware Interfaces

The system operates without specialized hardware dependencies, utilizing standard computing infrastructure for all operations. Input methods support conventional keyboard and mouse interactions with enhanced accessibility through keyboard navigation patterns and screen reader compatibility. Display requirements accommodate various resolutions and aspect ratios through responsive design principles, with particular attention to data visualization legibility across different screen sizes. No specialized peripherals, storage devices, or network equipment requirements exist beyond standard web hosting environments.

5.3 Software Interfaces

Frontend Interface Layer: The client-side implementation utilizes HTML5 semantic markup for structure, CSS3 with Flexbox/Grid layouts for responsive presentation, and JavaScript for interactive behaviors without framework dependencies. Chart visualization integrates Chart.js library through Canvas rendering with fallback options for compatibility scenarios.

Communication with backend services employs Fetch API with JSON data serialization, while local state management utilizes browser storage APIs for offline functionality support.

Backend Interface Layer: The server-side implementation employs PHP 8.2 with object-oriented architecture following modular design principles. The PDO abstraction layer provides database-agnostic data access with prepared statement security. Business logic encapsulation separates concerns between data validation, processing rules, and presentation preparation. API endpoints follow RESTful conventions with appropriate HTTP method utilization and status code responses.

Database Interface Layer: MySQL 8.0 serves as the persistent data store with InnoDB engine ensuring ACID compliance and referential integrity. Database schema implements normalized structure with appropriate indexing for performance optimization. Connection management includes connection pooling for efficiency and graceful degradation during high-load scenarios. Migration capabilities support schema evolution through version-controlled script execution.

5.4 Communications Interfaces

The system employs standard web protocols for all communications, with HTTP/HTTPS serving as the primary transport mechanism. Client-server interactions utilize JSON format for data exchange with consistent structure and error handling. Authentication employs session cookies with HttpOnly and Secure flags when available, while API interactions include CSRF token validation for state-changing operations.

Network communication assumes standard TCP/IP connectivity without specialized firewall configurations, while performance considerations include HTTP/2 support for multiplexed requests and compression for payload optimization.

Appendix A: Glossary

Table A.1: Terminology Definitions

Term	Full Expansion	Definition	Contextual Usage
FR	Functional Requirement	Specification defining system behaviors and operations	Used throughout SRS to identify mandatory system capabilities
NFR	Non-Functional Requirement	Specification defining system qualities and constraints	Applied to usability, security, performance, and reliability aspects
MVC	Model-View-Controller	Architectural pattern separating data, presentation, and control	Foundation for system structure ensuring maintainability
PDO	PHP Data Objects	Database abstraction layer providing secure data access	Primary mechanism for database interactions preventing SQL injection
SQL	Structured Query Language	Standard language for relational database management	Used for all database schema definitions and query operations
XSS	Cross-Site Scripting	Security vulnerability allowing client-side code injection	Mitigated through output encoding and input validation
UML	Unified Modeling Language	Standardized modeling language for software design	Employed for all system diagrams ensuring consistent notation
SUS	System Usability Scale	Standardized questionnaire for usability assessment	Primary metric for evaluating user experience quality
OWASP	Open Web Application Security Project	Community-focused organization improving software security	Framework for security requirement definition and testing

Appendix B: Issues List

Open Issues Requiring Resolution:

1. Mobile responsiveness optimization requiring additional breakpoint definitions for smaller screen devices ensuring interface usability across diverse mobile platforms.
2. Advanced chart customization options development for power users enabling personalized visualization parameters and export formats.
3. Batch transaction import functionality implementation supporting historical data migration from spreadsheet formats with validation and mapping capabilities.
4. Multi-currency support architecture design accommodating users with international financial activities requiring exchange rate integration.
5. Automated backup scheduling system development providing user-controlled data protection with configurable retention policies.

Recently Resolved Issues:

1. PDF chart rendering quality enhancement resolved through server-side image generation replacing client-side canvas capture ensuring consistent output.
2. Budget warning timing optimization addressed through real-time calculation triggers replacing batch processing improving user feedback immediacy.
3. Category deletion handling refinement implemented through soft delete logic with transaction reassignment preserving historical data integrity.
4. Session security enhancement completed through regeneration mechanisms and configurable timeout policies preventing fixation attacks.
5. Database performance optimization achieved through query restructuring and strategic indexing maintaining responsiveness with growing datasets.

Future Enhancement Considerations:

1. Mobile notification system integration providing budget alerts and transaction reminders through platform-specific push mechanisms.
2. Advanced analytics implementation incorporating predictive capabilities for spending patterns and financial forecasting models.
3. Multi-user collaboration features enabling shared household budgeting with permission controls and activity tracking.
4. External API development allowing third-party integrations with complementary financial services and tools.
5. Enhanced security mechanisms implementation including two-factor authentication options and advanced threat detection.

References

- ✓ IEEE Computer Society. (1998). *IEEE Standard 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. New York: IEEE.
- ✓ Pressman, R.S. (2020). Software Engineering: A Practitioner's Approach (9th ed.). New York: McGraw-Hill Education.
- ✓ OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/
- ✓ Oracle Corporation. (2025). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
- ✓ PHP Documentation Group. (2025). PHP: The Right Way. Retrieved from https://phptherightway.com/
- ✓ Object Management Group. (2017). Unified Modeling Language Specification Version 2.5.1. Retrieved from https://www.omg.org/spec/UML/
- ✓ Brooke, J. (1996). SUS: A "Quick and Dirty" Usability Scale. In P.W. Jordan, B. Thomas, B.A. Weerdmeester, & A.L. McClelland (Eds.), Usability Evaluation in Industry. London: Taylor & Francis.